Evidence Index

Enterprise DevSecOps Security Lab - Evidence Catalog and Validation Traceability

Document	Evidence Index Document
Project	Enterprise DevSecOps Security Lab
Author	Jagriti Banerjee
Environment	Private Ubuntu VM, Docker, Kind Kubernetes, GitHub Actions, ArgoCD, Falco, Prometheus, Grafana
Status	Portfolio-ready documentation

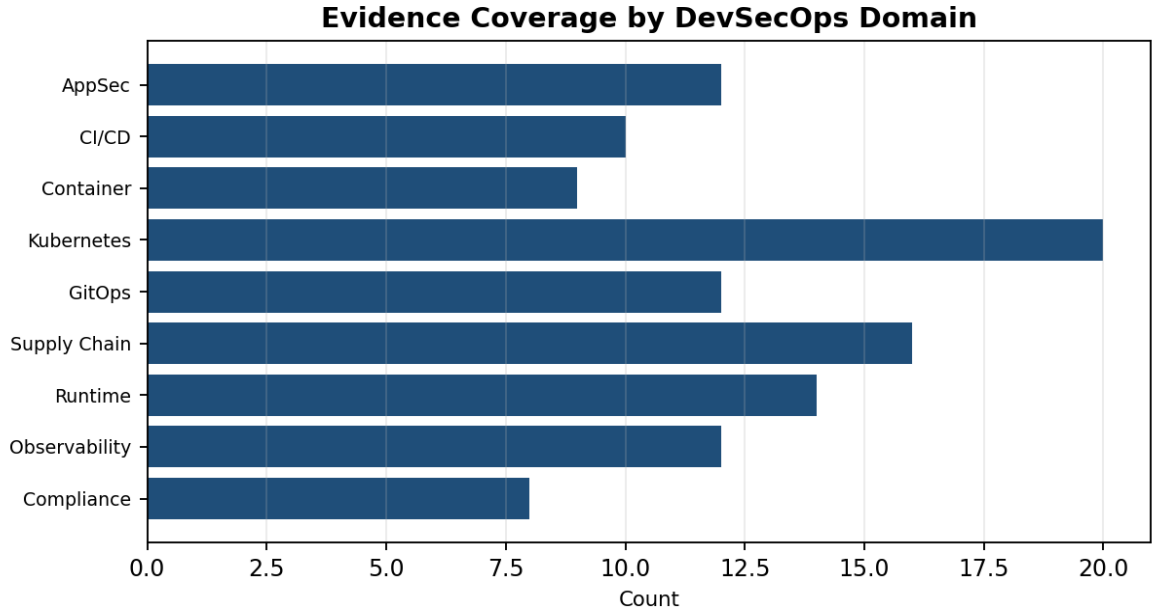
This document is written as professional portfolio evidence for a private enterprise-style DevSecOps lab. It is not a claim of formal certification, compliance attestation, or third-party audit approval.

Document quality controls: structured scope, evidence traceability, control mapping, validation criteria, residual risk notes, and executive-ready summary sections.

1. Executive Summary

This Evidence Index provides a traceable catalog of screenshots, reports, command outputs, and validation artifacts produced during the Enterprise DevSecOps Security Lab. The index is designed for recruiter review, audit-style walkthroughs, and repeatable project validation. It links each item to the relevant project phase, security control, risk, and proof objective.

The evidence set covers the full lifecycle: environment setup, vulnerable application testing, SAST/SCA, container hardening, Kubernetes controls, GitOps drift remediation, supply-chain security, runtime detection, observability, and compliance alignment.



2. Evidence Collection Standard

Evidence was organized using a consistent naming pattern, screenshot numbering, and artifact grouping. Each evidence item should be retained in the repository under evidence/screenshots or security/reports, with sensitive values such as tokens, passwords, and private keys removed or masked.

Requirement	Implementation in this Project
Traceability	Each screenshot is mapped to a project phase, control objective, and validation purpose.
Reproducibility	Command-based evidence includes enough context to repeat the validation in the private VM lab.
Confidentiality	No private Cosign key, GitHub PAT, kubeconfig secret value, or real production data should be committed.
Integrity	Generated reports, screenshots, and PDF documents are stored as immutable portfolio evidence after review.
Reviewability	Evidence items are grouped by domain so recruiters and reviewers can quickly understand the lab story.

3. Evidence Catalog

The following table is the primary evidence index. It can be copied into docs/evidence/EVIDENCE_INDEX.md and used as the central navigation file for screenshots and report artifacts.

ID	Artifact / Screenshot	Domain	What the Evidence Proves	Handling Notes
EV-001	01-ubuntu-endpoint-resources.png	Environment setup	Baseline VM RAM/CPU/disk; proves private lab context and resource constraints.	Retain screenshot; no secrets.
EV-002	02-swap-enabled-and-resource-check.png	Environment setup	Swap enabled and disk verified; supports reproducibility under limited resources.	Retain screenshot; no secrets.
EV-003	03-docker-hello-world.png	Container platform	Docker engine installed and operational.	Retain terminal proof.
EV-004	04-repo-structure-created.png	Repository structure	Professional folder layout for workflows, app code, Kubernetes manifests, reports, policies, and screenshots.	Retain tree output.
EV-005	05-vulnerable-app-running-local.png	AppSec baseline	Flask application reachable locally through health endpoint.	Retain app health proof.
EV-006	06-sql-injection-proof.png	Vulnerability proof	SQL injection returns unauthorized data through unsafe query construction.	Retain with no sensitive real data.
EV-007	07-command-injection-proof.png	Vulnerability proof	Command injection proof using controlled lab endpoint and benign command output.	Retain sanitized output.
EV-008	08-bandit-sast-findings.png	SAST evidence	Bandit identifies insecure subprocess/template/string-construction patterns.	Retain scan output.
EV-009	09-pip-audit-dependency-findings.png	SCA evidence	pip-audit identifies vulnerable Python dependency versions.	Retain scan output.
EV-010	10-docker-image-built.png	Container build	Application image built successfully.	Retain image ID/tag.
EV-011	12-trivy-image-scan.png	Container security	Trivy image scan results collected.	Retain report artifact.
EV-012	22-docker-image-non-root-user.png	Container hardening	Container runtime UID/GID proves non-root execution.	Retain id output.
EV-013	23-kind-cluster-with-calico-running.png	Kubernetes platform	Kind cluster running with Calico for NetworkPolicy enforcement.	Retain node/CNI proof.
EV-014	25-secure-namespace-created.png	Pod Security	Restricted namespace labels applied.	Retain namespace labels.
EV-015	27-rbac-can-read-configmaps.png	RBAC validation	Dedicated ServiceAccount can perform intended low-risk action.	Retain can-i output.
EV-016	28-rbac-cannot-read-secrets.png	RBAC validation	ServiceAccount cannot read secrets.	Retain can-i output.
EV-017	31-demo-app-pod-running.png	Kubernetes deployment	Hardened application pod is ready.	Retain pod status.
EV-018	35-network-policies-created.png	Network segmentation	Default-deny and allow-list NetworkPolicies created.	Retain networkpolicy list.
EV-019	37-networkpolicy-blocked-untrusted-pod.png	Network validation	Untrusted pod is blocked from calling app.	Retain timeout/block proof.
EV-020	38-networkpolicy-allowed-trusted-pod.png	Network validation	Allowed pod can reach app service.	Retain curl success.
EV-021	40-readonly-filesystem-proof.png	Container hardening	Read-only root filesystem prevents write to /app while /tmp remains usable.	Retain command output.
EV-022	41-trivy-k8s-manifest-scan.png	IaC scanning	Kubernetes manifests scanned for security misconfiguration.	Retain Trivy config report.
EV-023	47-argocd-app-synced-healthy.png	GitOps	ArgoCD application is synced and healthy.	Retain UI/CLI proof.
EV-024	53-argocd-self-healed-back-to-git-state.png	Drift remediation	ArgoCD self-heals manual replica drift.	Retain before/after.

ID	Artifact / Screenshot	Domain	What the Evidence Proves	Marking Notes
EV-025	60-image-pushed-to-ghcr.png	Registry	Image pushed to GitHub Container Registry.	Retain registry output.
EV-026	64-cosign-signature-verified.png	Supply chain	Cosign verification succeeds against public key.	Retain verification output.
EV-027	66-sbom-generated-with-syft.png	SBOM	Syft generates SPDX/CycloneDX SBOM artifacts.	Retain package count.
EV-028	68-sbom-attestation-verified.png	Supply chain attestation	SBOM attestation verification succeeds.	Retain verify-attestation output.
EV-029	70-slsa-provenance-attestation-verified.png	Provenance	SLSA-style provenance attestation verified.	Retain attestation output.
EV-030	75-gatekeeper-blocked-bad-deployment.png	Admission control	OPA Gatekeeper blocks non-compliant Deployment.	Retain denial output.
EV-031	79-falco-installed-running.png	Runtime detection	Falco running in cluster.	Retain pod status.
EV-032	84-container-escape-chroot-proof.png	Attack simulation	Privileged hostPath pod demonstrates host filesystem access pattern.	Retain controlled lab proof.
EV-033	85-falco-detected-privileged-pod-activity.png	Detection evidence	Falco alerts on suspicious privileged/sensitive file activity.	Retain alert output.
EV-034	87-privileged-pod-blocked-by-pod-security.png	Preventive control	Pod Security restricted blocks the privileged pod.	Retain admission denial.
EV-035	89-rbac-overprivileged-sa-can-get-secrets.png	RBAC attack proof	Overprivileged ServiceAccount can access secrets before fix.	Retain can-i output.
EV-036	92-rbac-fixed-cannot-get-secrets.png	RBAC remediation	Least-privilege fix blocks secret access.	Retain can-i output.
EV-037	95-prometheus-grafana-installed.png	Observability	Prometheus and Grafana deployed.	Retain pod/service status.
EV-038	103-prometheus-falco-targets-up.png	Metrics integration	Prometheus scrapes Falco/Falcosidekick targets.	Retain target health.
EV-039	107-custom-falco-security-dashboard-imported.png	Dashboarding	Grafana Falco runtime security dashboard imported.	Retain dashboard screenshot.
EV-040	109-falco-prometheus-alert-rules.png	Alerting	Prometheus rules for Falco detections are present.	Retain rules page.

4. Evidence Grouping by Project Phase

Phase	Evidence IDs	Reviewer Narrative
Private VM and repository setup	EV-001 to EV-004	Shows that the project was created in an isolated private lab with a structured repository ready for enterprise-style documentation.
Vulnerable application and AppSec findings	EV-005 to EV-009	Demonstrates intentional vulnerable Flask routes, exploit proof, Bandit SAST, and pip-audit dependency scanning.
Container build and hardening	EV-010 to EV-012	Shows that the app was containerized, scanned, and later executed as a non-root user.
Kubernetes hardening and network controls	EV-013 to EV-022	Proves namespace security, RBAC least privilege, NetworkPolicy allow/deny behavior, read-only filesystem, and IaC scanning.
GitOps	EV-023 to EV-024	Proves ArgoCD sync, health, drift detection, and self-healing.
Supply chain security	EV-025 to EV-030	Proves GHCR publishing, Cosign signature verification, Syft SBOM generation, attestations, provenance, and Gatekeeper admission control.
Runtime detection and response	EV-031 to EV-036	Shows Falco running, attack simulation, detection, Pod Security remediation, and RBAC privilege remediation.
Observability	EV-037 to EV-040	Shows Prometheus/Grafana integration, Falco metric targets, dashboarding, and alert rules.

5. Evidence Quality Checklist

• Screenshots should include enough terminal context to identify the command and result without exposing secrets.

• Reports should be saved in security/reports with clear names such as bandit-report.json, trivy-image-report.txt, and falco-live-alerts.txt.

• For every critical control, retain both the failure demonstration and the remediation validation proof.

• Any evidence containing private tokens, Cosign private keys, generated passwords, or registry credentials must be excluded from Git and final PDFs.

• Evidence screenshots should be referenced from README, SECURITY_FINDINGS_REPORT, and compliance documents using consistent filenames.

6. Recommended Evidence Index File Structure

The following folder structure is recommended for the final GitHub repository. The screenshot names may be adjusted to match the files collected in the private VM.

```
evidence/screenshots/
security/reports/
security/sbom/
security/provenance/
security/policies/
monitoring/dashboards/
monitoring/rules/
docs/evidence/EVIDENCE_INDEX.md
```

7. Evidence Gaps and Next Actions

Gap / Improvement	Reason	Recommended Action
Formal screenshot naming policy	Improves reviewer navigation and reduces ambiguity.	Rename screenshots with zero-padded IDs and keep a matching index table.
Retest command transcript	Shows exact commands used after remediation.	Add terminal transcript files under security/reports/retest/.

Gap / Improvement	Reason	Recommended Action
Code diff evidence	Shows before and after remediation.	Add sanitized Git diff snippets for SQL injection, command injection, and unsafe rendering fixes.
Signed release bundle	Strengthens supply-chain story.	Attach SBOM and provenance to a versioned image tag rather than only latest.
Dashboard export evidence	Preserves observability setup.	Commit Grafana JSON and PrometheusRule YAML with screenshots of successful import.

8. Chain-of-Custody and Review Checklist

Review Item	Expected Result
File naming	Each screenshot or report uses a predictable, numbered filename tied to the evidence index.
Scope statement	The README and report clearly state that testing was performed only in a private lab.
Secret review	No GitHub PAT, Cosign private key, ArgoCD admin password, kubeconfig credential, or token appears in screenshots or committed files.
Artifact retention	JSON, TXT, SARIF, SBOM, provenance, dashboard JSON, and PrometheusRule YAML files are committed when safe.
Reviewer path	A reviewer can move from README to findings report to evidence index without guessing where proof is stored.

9. Recommended Source Paths

Path	Purpose
evidence/screenshots/	Human-readable screenshots for terminal, browser, dashboard, and UI evidence.
security/reports/	Scanner outputs, Falco logs, Trivy reports, pip-audit reports, Bandit reports, and retest artifacts.
security/sbom/	SPDX and CycloneDX SBOM files generated by Syft.
security/provenance/	SLSA-style provenance predicate and related attestation evidence.
monitoring/dashboards/	Grafana dashboard JSON exports.
monitoring/rules/	PrometheusRule YAML alert definitions.
docs/evidence/	Evidence index and cross-reference documents.